

CODEHIVE 2K26

Event Registration Platform

Production-ready technical stack, system architecture, database design, security model, and phased development workflow.

DOCUMENT	VERSION	STATUS
Technical Blueprint	1.0	Implementation Ready

Recommended architecture: Next.js 16 + TypeScript + Tailwind CSS v4 + shadcn/ui + Prisma + PostgreSQL/Neon + Cloudinary + Clerk + Resend + QR Check-in + Vercel.

What we are building

A complete event operations platform for public registration, event discovery, participant management, admin operations, communication, and on-site QR check-in.

PUBLIC EXPERIENCE	ADMIN EXPERIENCE	EVENT OPERATIONS
Landing page Event listing Event details Registration flow Confirmation page	Dashboard Registrations Participants Events Check-ins Reports	Registration IDs QR codes Email confirmations Check-in validation Exportable data

Core engineering principle

One full-stack Next.js application. Avoid an unnecessary Express backend. Next.js Server Actions handle secure mutations, Route Handlers expose required APIs, Prisma handles data access, and PostgreSQL remains the source of truth.

Success criteria

- A participant can discover an event and complete registration with clear validation.
- Every successful registration receives a unique registration number and QR code.
- Admins can search, filter, inspect, export, and manage registrations.
- On-site staff can scan/validate QR codes and prevent duplicate check-ins.
- Images and event media are stored in Cloudinary; the database stores metadata and secure URLs.
- The application is deployable to Vercel with production environment variables and database migrations.

The production stack

A modern TypeScript-first stack selected for fast development, maintainability, deployment simplicity, and strong integration between application layers.

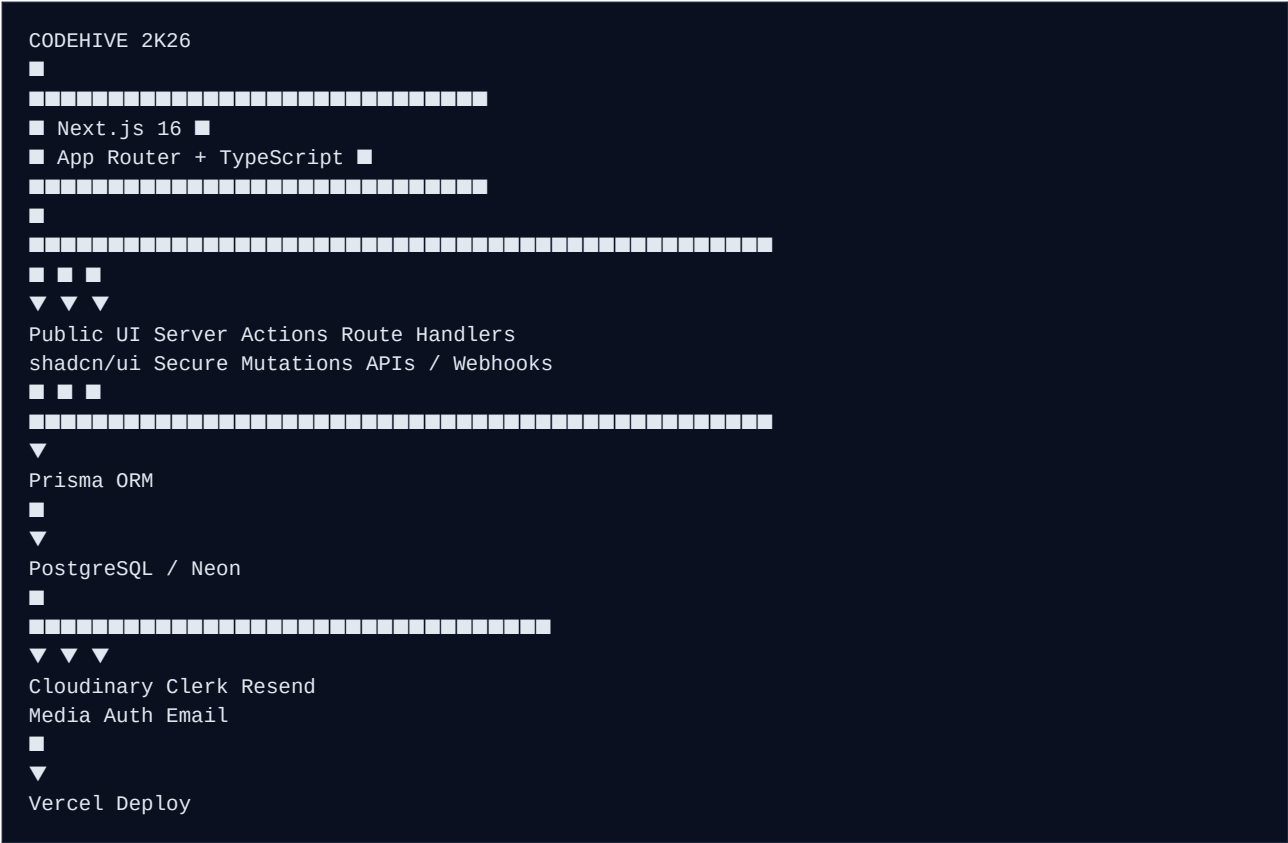
LAYER	TECHNOLOGY	ROLE
Framework	Next.js 16 + App Router	Full-stack application, routing, rendering, Server Actions, Route Handlers
Language	TypeScript	Type-safe application and database integration
UI	Tailwind CSS v4 + shadcn/ui	Design system, responsive layout, accessible components
Motion	Framer Motion	Subtle transitions and interaction feedback
Forms	React Hook Form + Zod	Form state and schema validation
Backend	Server Actions + Route Handlers	Mutations, APIs, webhooks, secure server logic
ORM	Prisma	Type-safe PostgreSQL access and migrations
Database	PostgreSQL / Neon	Registrations, participants, events, payments, check-ins
Auth	Clerk	Admin/organizer authentication and session management
Media	Cloudinary	Images, posters, logos, certificates and media delivery
Email	Resend	Registration confirmation and event communication
QR	qrcode + scanner library	Registration QR generation and check-in
Deployment	Vercel	Application deployment and preview environments
Package Manager	pnpm	Fast, efficient dependency management

Recommended baseline

Next.js 16 • TypeScript • App Router • pnpm
Tailwind CSS v4 • shadcn/ui • Framer Motion
React Hook Form • Zod • Prisma • PostgreSQL/Neon
Clerk • Cloudinary • Resend • QR Check-in • Vercel

How the application works

Keep the architecture intentionally simple: the browser talks to Next.js; server-side logic talks to the database and external services.



Request lifecycle

1. USER	2. NEXT.JS	3. VALIDATION	4. DATABASE	5. SERVICES
Fills registration	Server Action receives request	Zod validates input	Prisma transaction saves data	Cloudinary / Resend / QR

Rule: business-critical operations should be executed server-side. Never expose database credentials, Cloudinary API secrets, or privileged admin logic to the browser.

04 • DEVELOPMENT WORKFLOW

Phase-by-phase implementation plan

Build vertically: each phase should leave the application in a runnable state.

PHASE 01

Foundation & Project Setup

- Create Next.js project with pnpm
- Configure TypeScript, ESLint and Tailwind
- Install shadcn/ui and base components
- Set environment variable strategy
- Initialize Git + branch workflow

Deliverable: Clean deployable application shell

PHASE 02

Design System & Public Website

- Build navbar, footer and responsive layout
- Create CodeHive 2K26 branding
- Landing page + event highlights
- Event listing and details
- Mobile-first accessibility pass

Deliverable: Complete public-facing experience

PHASE 03

Database & Data Model

- Configure PostgreSQL/Neon
- Initialize Prisma
- Design Event / Participant / Registration models
- Run migrations
- Seed development data

Deliverable: Stable database foundation

PHASE 04

Registration Engine

- Build registration form
- React Hook Form + Zod validation
- Server Action for registration
- Unique registration number
- Success page + registration lookup

Deliverable: End-to-end registration

PHASE 05

Cloudinary & Email

- Configure Cloudinary upload
- Store media metadata in DB
- Generate confirmation email
- Add QR code generation
- Verify failure/retry handling

Deliverable: Media + communication pipeline

PHASE 06**Admin Dashboard**

- Clerk admin authentication
- Dashboard metrics
- Registration table
- Search/filter/sort
- Participant and event management
- CSV export

Deliverable: Operational admin console

PHASE 07**QR Check-in**

- Build scanner/check-in screen
- Validate registration status
- Prevent duplicate check-in
- Record timestamp and staff identity
- Show success/error feedback

Deliverable: Event-day check-in system

PHASE 08**Security, Testing & Production**

- Authorization audit
- Input and upload validation
- Rate-limit sensitive endpoints
- Test edge cases
- Production migration + deployment
- Monitoring and rollback plan

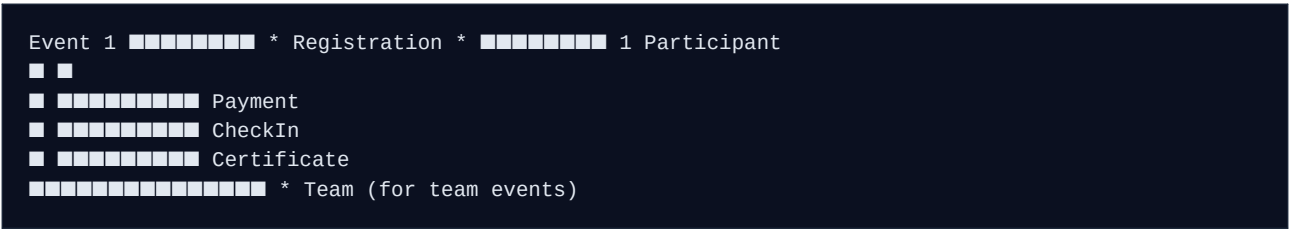
Deliverable: Production-ready release

Core data model

PostgreSQL is the source of truth. Keep files out of the database; store Cloudinary identifiers and URLs as metadata.

MODEL	KEY FIELDS	RESPONSIBILITY
Event	id, name, slug, description, venue, startAt, endAt, capacity	Event catalogue and scheduling
Participant	id, name, email, phone, college, department, year, imageUrl	Person registering for events
Registration	id, registrationNumber, participantId, eventId, status, checkedIn	Registration lifecycle and check-in state
Team	id, name, eventId, leaderId	Optional team-based events
Payment	id, registrationId, amount, status, providerRef	Paid event registration if required
CheckIn	id, registrationId, staffId, checkedInAt	Auditable event-day attendance
Certificate	id, participantId, eventId, certificateUrl	Certificate generation/delivery
Admin/User	id, role, authProviderId	Organizer/admin authorization

Relationship map

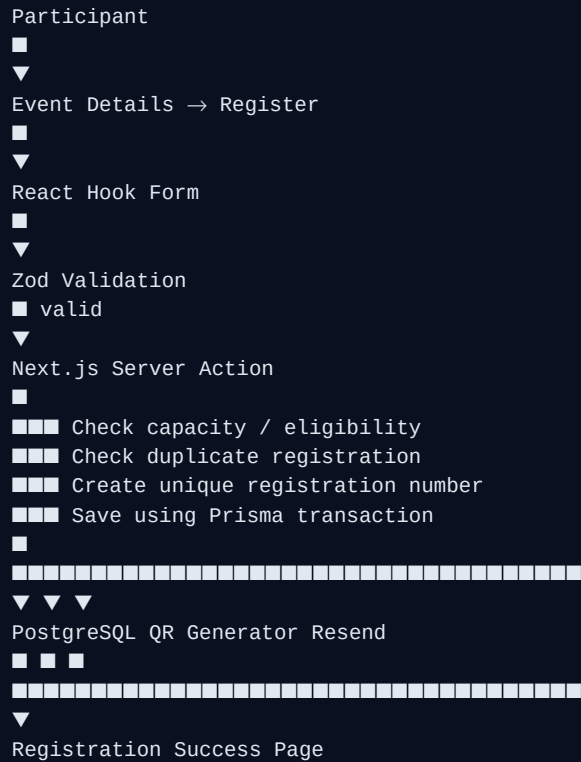


Registration status model

PENDING	CONFIRMED	CANCELLED	CHECKED-IN
Created but not fully confirmed	Valid registration	Registration invalidated/cancelled	Confirmed registration has attended

The critical business flow

Build and test this flow before adding dashboards and secondary features.



Validation rules to implement

- Required fields must be validated on the server even if client-side validation exists.
- Normalize email and phone values before duplicate checks.
- Use database uniqueness constraints for critical invariants.
- Do not trust event IDs, prices, roles, or status values coming from the client.
- Use a transaction for multi-step writes that must remain consistent.

Registration number format

CH26-XXXXXX

Example:

CH26-8F3K21

Storage architecture

Cloudinary handles media storage and delivery. PostgreSQL stores references, not binary files.

MEDIA	STORAGE	DATABASE METADATA
Participant photo	Cloudinary	imageUrl, publicId
Event poster	Cloudinary	eventId, imageUrl, publicId
Sponsor logo	Cloudinary	sponsorId, imageUrl, publicId
Certificate	Cloudinary or generated file storage	certificateUrl, participantId, eventId

Upload security

- Use server-controlled upload signatures or a secure server-side upload flow.
- Validate MIME type and file size before accepting uploads.
- Restrict upload folders by purpose, e.g. /codehive/participants and /codehive/events.
- Never expose CLOUDINARY_API_SECRET in client-side code or NEXT_PUBLIC environment variables.
- Save the Cloudinary public ID so assets can later be replaced or deleted safely.

Recommended environment variables

```
DATABASE_URL=  
DIRECT_URL=  
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY=  
CLERK_SECRET_KEY=  
CLOUDINARY_CLOUD_NAME=  
CLOUDINARY_API_KEY=  
CLOUDINARY_API_SECRET=  
RESEND_API_KEY=  
NEXT_PUBLIC_APP_URL=
```

08 • ADMIN & OPERATIONS

Dashboard structure

The admin area should optimize for event staff: fast search, clear status, safe actions, and useful operational metrics.

PAGE	KEY FEATURES	PRIMARY USERS
Dashboard	Registration count, event count, check-ins, recent activity	Organizer / Admin
Registrations	Search, filter, sort, view details, status updates, export	Organizer / Admin
Participants	Profile, registrations, contact data, media	Organizer / Admin
Events	Create/edit event, capacity, schedule, venue	Admin
Check-in	Scan QR, validate, prevent duplicate check-in	Event Staff
Reports	CSV export, attendance, event-wise counts	Admin
Settings	Event configuration, roles, operational settings	Admin

Role model

ROLE	ACCESS
SUPER_ADMIN	All settings, users, events, registrations and exports
ORGANIZER	Event and registration management; operational dashboard
STAFF	QR check-in and limited participant lookup

09 • SECURITY & RELIABILITY

Production checklist

Security is part of the architecture, not a final cosmetic step.

AREA	REQUIREMENT
Authentication	Use Clerk for admin/organizer authentication; protect /admin routes.
Authorization	Check role/permission server-side for every privileged mutation.
Validation	Validate all external input with Zod on the server.
Database	Use Prisma transactions and unique constraints for critical invariants.
Uploads	Validate file type/size and keep Cloudinary secrets server-only.
Secrets	Store credentials in environment variables; never commit .env files.
Rate limits	Rate-limit registration, QR validation and sensitive APIs.
Audit	Record important admin/check-in actions with actor and timestamp.
Backups	Use managed PostgreSQL backups and document recovery procedures.
Errors	Return safe user-facing errors; do not leak stack traces or secrets.

Testing matrix

TEST TYPE	WHAT TO COVER
Unit	Validation, registration ID generation, status transitions
Integration	Server Action + Prisma + transaction behavior
E2E	Register → confirmation → lookup → QR check-in
Security	Unauthorized admin access, invalid IDs, malicious uploads, duplicate requests
Load	Registration spikes before deadlines and event-day QR scanning

10 • DEPLOYMENT

Development → Preview → Production

Use Git-based deployment so every meaningful change is reviewable and reversible.

STAGE	FLOW	PURPOSE
Local	pnpm dev → local development → test	Fast implementation
Preview	Git push → Vercel Preview → integration/E2E testing	Review changes safely
Production	Merge to main → migration → Vercel deployment	Live CodeHive platform

Git workflow

```
main → production
develop → integration
feature/registration → registration work
feature/admin → admin dashboard
feature/checkin → QR check-in

feature/* → PR → review → develop → main
```

Production launch checklist

- Production database configured and migration tested.
- All production environment variables configured in Vercel.
- Cloudinary upload and deletion flows verified.
- Resend sender/domain configuration verified.
- Clerk production instance and authorized roles verified.
- Admin authorization tested with non-admin accounts.
- Registration duplicate handling tested.
- QR check-in duplicate prevention tested.
- Mobile layout tested on registration and check-in screens.
- Backup and rollback procedure documented.

11 • CODEBASE STRUCTURE

Recommended project organization

Keep domain logic separate from UI components and infrastructure clients.

```
codehive2k26/
├── app/
│   ├── (public)/
│   │   ├── page.tsx
│   │   ├── events/
│   │   ├── register/
│   │   ├── registration/
│   │   ├── admin/
│   │   ├── dashboard/
│   │   ├── registrations/
│   │   ├── participants/
│   │   ├── events/
│   │   ├── check-in/
│   │   ├── api/
│   │   ├── registrations/
│   │   ├── check-in/
│   │   ├── webhooks/
│   │   ├── actions/
│   │   ├── registration.ts
│   │   ├── participant.ts
│   │   ├── event.ts
│   │   ├── check-in.ts
│   │   ├── components/
│   │   ├── ui/
│   │   ├── registration/
│   │   ├── events/
│   │   ├── admin/
│   │   ├── lib/
│   │   ├── prisma.ts
│   │   ├── cloudinary.ts
│   │   ├── auth.ts
│   │   ├── email.ts
│   │   ├── qr.ts
│   │   ├── validations.ts
│   │   ├── prisma/
│   │   ├── schema.prisma
│   │   ├── public/
│   │   ├── types/
│   │   ├── hooks/
│   │   ├── utils/
│   │   ├── next.config.ts
│   │   ├── package.json
│   │   ├── pnpm-lock.yaml
```

Implementation rule

UI components should not contain database queries. Put data mutations in Server Actions or dedicated server modules, validate inputs centrally, and keep external service clients isolated under **lib/**.

Build order

The order below minimizes rework and gets the highest-risk business flow working early.

ORDER	MILESTONE	EXIT CONDITION
01	Project foundation	Next.js app runs; lint/build pass; Git initialized
02	Public event website	Responsive landing, events and details complete
03	Database	Prisma schema migrated and seeded
04	Registration engine	Real registration can be created and retrieved
05	Media + email + QR	Confirmation package generated reliably
06	Admin dashboard	Staff can operate registrations without DB access
07	Check-in	QR scan validates and records attendance
08	Hardening	Security, tests, mobile QA, deployment checklist complete

Final architecture snapshot

FRONTEND	BACKEND	DATA	INFRA
Next.js Tailwind shadcn/ui Framer Motion	Server Actions Route Handlers Zod	Prisma PostgreSQL/Neon Cloudinary	Clerk Resend Vercel pnpm

CodeHive 2K26 — Engineering principle: keep the core registration path reliable, auditable, secure, and simple. Add complexity only when a real event requirement demands it.